

Spark Homes Repair Estimator

Contest Submission - Priyank Mandal

1. Most Interesting Design/UX Decision

Ditching full-DOM rebuilds for surgical updates. In a single-file app with no framework, the easy and tempting path is innerHTML teardowns on every state change. You save state, you rebuild, you move on. But on a mid-range Android phone with 20 plus room instances, each containing a dozen line items, that means rebuilding hundreds of DOM nodes just to flip one checkbox. The screen visibly flashes. Scroll position resets to the top. Any text input the user was mid-sentence in gets dropped. The person in the field, already frustrated because the house has knob-and-tube wiring and the roof is leaking, now also hates the app. So I made toggleItem, setQty, and setPrice update only the affected row class, line total, group badge, progress bar, per-section cost breakdown, and grand total. Nothing else gets touched. The DOM stays put. Scroll position survives. Focus state survives. Whatever half-finished number the user typed into a quantity field survives. This was the difference between an app that feels like butter and one that feels like sandpaper, and it cost about 40 lines of carefully targeted JavaScript. The key insight was using scoped item keys (roomId::itemId) as the ID prefix for every DOM element, so no matter how many rooms and items exist, the update functions can find and patch individual cells with a single getElementById call. No traversal, no queries, no loops over irrelevant nodes. It also meant the Playwright tests stopped fighting stale element handles on rapid interactions, which was a happy side effect I did not anticipate but deeply appreciated.

2. What is Broken or Fragile

Custom items are stored in a flat array and referenced by their index in that array. Delete one and everything after it shifts like a precarious Jenga tower. It works because the event delegation system reads live indices from data attributes on every interaction, so it always grabs the correct item even after deletions. But the array-index pattern is held together by duct tape and good intentions. Every time I touch that code I hold my breath a little. A UUID-keyed map would be the grownup fix, and if I had another day I would refactor it. The cost breakdown section has a minor rendering bug: it only appears in the DOM during a full render pass, not during surgical updates. If you check the very first item of a brand new project, the breakdown panel does not exist yet, so the per-section total update targets null references and silently fails. It pops into existence as soon as you trigger a full render by adding a room, switching tabs, or importing a CSV. It happens once per project and the numbers are still saved correctly, but a sharp-eyed user would notice the card is missing for those first few clicks. Photo capture on iOS has a known Safari limitation: the capture attribute on file inputs is a suggestion, not a command, so sometimes the camera app does not launch and you get a file picker instead. This is Apple being Apple, and there is no JavaScript-only fix for it. A native wrapper would help, but that is outside the scope of this project.

3. Creative Addition - Deal Analyzer

The core app tells you how much a repair costs. That is table stakes. The Deal Analyzer tells you whether making that repair is worth it. Knowing the repairs cost \$45K means nothing in isolation. Knowing they cost \$45K on a house with a \$200K ARV and a \$120K purchase price, after \$8K in closing costs at 3 percent, \$7K in monthly holding costs times 4 months, and \$12K in selling costs at 6 percent of ARV, leaving a \$15K profit and a 10.5 percent ROI? That is a decision you can take to the bank, or to your investor. The Deal Analyzer sits below the grand total and computes the full cost basis by combining purchase price, closing costs (as a percentage), total holding cost (monthly cost times months held), selling costs (as a percentage of ARV), and the repair estimate. It outputs total cost basis, estimated profit, ROI percentage, and margin percentage. All of them are color coded: green text for profit, red for loss, and the inputs have a clean amber border when focused. Every input persists per-project in localStorage along with the line items and rooms, so you do not retype the numbers when the page refreshes. This is critical for the field use case: an agent might open the app, lose signal, close it, and come back later without the ARV number memorized. It just remembers. The secondary creative addition is CSV price list import. Agents can upload the official Pricing List.csv from the contest briefing to bulk update all 108 standard item prices at once. This directly addresses the global pricing requirement from the brief and means you never have to manually edit prices one by one when material costs change. The import is a one-click file picker that parses the CSV, matches items by ID (the ig-XX keys), updates the master price list, and re-renders everything to show the new totals. It works offline because the file is read client-side.

4. What I Would Ship Next (Two More Days)

Three things. First, photo-to-item association. Right now photos are attached to projects, not to rooms or line items. An agent takes a photo of a cracked foundation but there is no way to link that photo to the "Foundation Crack Repair" line item in the export. I would add a lightweight annotation system: tap a photo, pick a room and item from a dropdown, and the export CSV/ZIP would include a column showing which photo documents which repair. Second, JSON export and import for transferring estimates between devices without any backend. Imagine an agent starts a walkthrough on their phone and emails the JSON file to a colleague who opens it on a tablet at the office. Today that means starting from scratch on the second device. The JSON format would be a simple serialization of the project data model, maybe a hundred lines of code to implement both export and import. Third, a printable PDF scope of work summary, separate from the contest writeup. Contractors love paper. They write on it. They lose it. They ask for another copy. A clean one-page PDF with checked items, quantities, unit prices, line totals, and section subtotals, printed in a font large enough to read on a dusty job site with poor lighting, would save everyone a headache. It would also make the app feel more professional when handed to a client or investor during a walkthrough.

5. Role of AI Tools

Claude Code was used throughout development for the following: generating the 108-item data model by parsing the Pricing List CSV from the contest brief into the JavaScript object literal with groups, rooms, and default quantities; writing the IndexedDB photo storage wrapper that handles compression, store, retrieve, and delete operations; implementing the inline ZIP generator and the minimal XLSX builder (a hand-coded XML spreadsheet inside a ZIP, no external libraries required for the fallback); debugging the service worker caching strategy to handle flaky CDN connections with Promise.allSettled instead of Promise.all; writing the Playwright test suite that covers project creation, item toggling, room management, tab switching, CSV import, and photo capture; and generating this writeup. The overall architecture of the app, the event delegation system that replaced 20-plus inline event handlers with a single listener on the app container, the room and group data model with scoped item keys, the surgical DOM update layer that targets individual cells by derived IDs, and the deal analyzer logic and math were all designed and implemented by me. AI was most useful for catching edge cases in the event delegation router (like the way change events bubble differently than click events on checkboxes) and for rapidly iterating on the service worker so it handles a slow or flaky network gracefully instead of bricking the entire install on the first CDN timeout. I would estimate AI wrote roughly 30 percent of the final code. I made sure the 70 percent that matters actually works on a phone in a house with bad reception.